

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



MICROCONTROLLERS - CO3010

Lab Session Report

LAB 01 REPORT LED ANIMATIONS

Advisor(s): PHAN VAN SY

Student(s): Nguyen Minh Phuc 2453005

HO CHI MINH CITY, SEPTEMBER 2026



Contents

1	Setup and Signal Convention	4
2	Basic LED Control and Traffic Systems (Exercises 1–5)	5
2.1	Exercise 1: Two-LED Alternation	5
2.2	Exercise 2: Three-LED Traffic Signal	7
2.3	Exercise 3: Four-Way 12-LED Traffic Intersection	9
2.4	Exercise 4: 7-Segment Common-Anode Display Decoding	11
2.5	Exercise 5: Integrated Traffic Light with Countdown Display	14
3	Analog Clock Simulation with 12 LEDs (Exercises 6–10)	17
3.1	Exercise 6: 12-LED Clock Dial Interface and Sequential Test	17
3.2	Exercise 7: Complete <code>clearAllClock()</code> Implementation	20
3.3	Exercise 8: Complete <code>setNumberOnClock(int num)</code> Implementation	20
3.4	Exercise 9: Complete <code>clearNumberOnClock(int num)</code> Implementation	21
3.5	Exercise 10: Integrated Real-Time 12-LED Analog Clock	21
4	Repository Artifact Links	24

List of Figures

2.1	Exercise 1 Proteus schematic: Two-LED alternation	6
2.2	Exercise 2 Proteus schematic: Three-LED traffic signal	8
2.3	Exercise 3 Proteus schematic: 12-LED four-way intersection	10
2.4	Exercise 4 Proteus schematic with 7-segment display	12
3.1	Exercise 6 Proteus schematic: 12-LED analog clock on PA4–PA15	18
3.2	12-LED analog clock dial schematic and hand vector representation	22

List of Tables

2.1	Exercise 1 pin states and timing alternation	5
2.2	Exercise 2 traffic phase states and timing	7
2.3	Exercise 3 GPIO pin assignments	9
2.4	Four-way traffic light state transition explanation	9



2.5	Segment encoding truth table for common-anode 7-segment display	11
3.1	Exercise 6 dial LED pin assignments and sweep sequence	17
3.2	Clock dial numeral to STM32 GPIO pin mapping	22

Listings

2.1	Infinite loop for Exercise 1	5
2.2	Infinite loop for Exercise 2	7
2.3	display7SEG() function	13
2.4	Countdown helper function runPhase()	14
2.5	Integrated traffic countdown while loop	15
3.1	Sequential LED test program	19
3.2	clearAllClock() function	20
3.3	setNumberOnClock() function	20
3.4	clearNumberOnClock() function	21
3.5	Clock display and atomic BSR driver	23
3.6	Real-time tick loop with rollover	24



1 Setup and Signal Convention

Lab 01 uses the **STM32F103C6**, programmed in STM32CubeIDE and represented in Proteus, to complete the LED, traffic-signal, seven-segment, and 12-LED clock exercises in the laboratory manual^[4]. The firmware and Proteus project files are available in the [course repository at commit 77b62e4](#). Each exercise below gives its relevant pin assignments, schematic or source code, and expected behavior.

The simulated LEDs and common-anode seven-segment display use **active-low control** from a +3.3 V supply: `GPIO_PIN_RESET` drives the cathode low and turns an indicator **ON**; `GPIO_PIN_SET` drives it high and turns the indicator **OFF**. The pin-state tables and code descriptions use this convention. Matching numbered connection labels in the Proteus captures identify the same net.



2 Basic LED Control and Traffic Systems (Exercises 1–5)

2.1 Exercise 1: Two-LED Alternation

Objective and Circuit Setup Extend the baseline blinky project by adding a second LED (LED-YELLOW) connected to pin PA6. The red LED remains on PA5. Both LEDs are wired in active-low mode. The two LEDs alternate states every 2 seconds: when Red is ON, Yellow is OFF, and vice versa (Table 2.1). The complete circuit schematic is presented on the subsequent dedicated page (Figure 2.1).

Phase / Sub-cycle	Duration	PA5 (Red)	PA6 (Yellow)	Visual State
Phase A	2000 ms	RESET (0 V)	SET (3.3 V)	Red ON, Yellow OFF
Phase B	2000 ms	SET (3.3 V)	RESET (0 V)	Red OFF, Yellow ON

Table 2.1: Exercise 1 pin states and timing alternation

Firmware Implementation The implementation executes within the infinite `while` (1) loop in `main.c`. The listing below presents the verbatim source code from `Lab1_e1/Core/Src/main.c`:

```
1  /* Infinite loop */
2  /* USER CODE BEGIN WHILE */
3  while (1)
4  {
5      // red on yellow off
6      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
7      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
8      HAL_Delay(2000);
9      // red off yellow on
10     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
11     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
12     HAL_Delay(2000);
13     /* USER CODE END WHILE */
14
15     /* USER CODE BEGIN 3 */
16 }
```

Listing 2.1: Infinite loop implementation for Exercise 1 (Lab1_e1/Core/Src/main.c)

Operational Behavior The source code dictates that the outputs alternate every 2000 ms, producing a nominal 4-second period with a 50% duty cycle per diode.

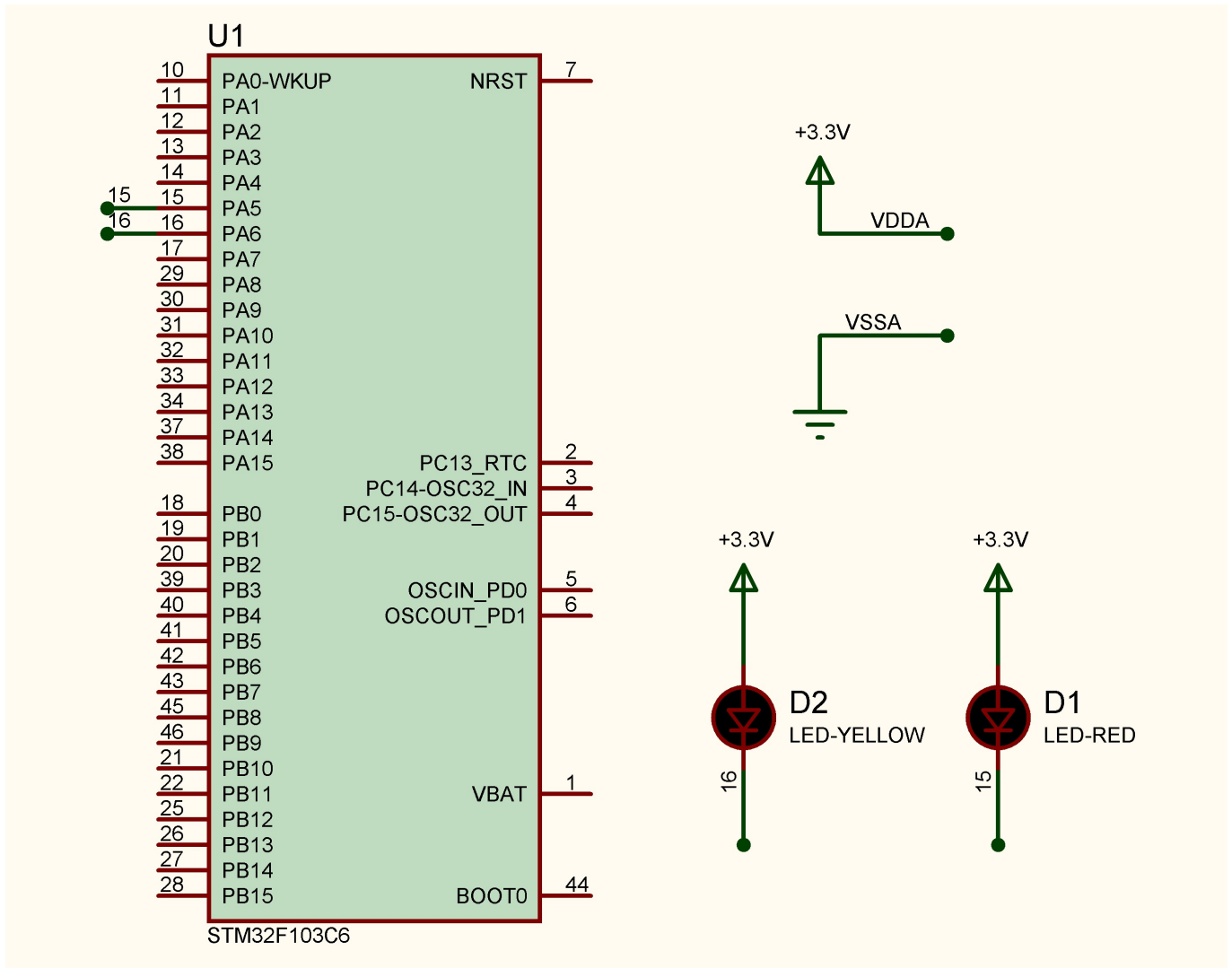


Figure 2.1: Exercise 1 Proteus schematic: Two-LED alternation ([Lab1_e1.pdsprj](#))



2.2 Exercise 2: Three-LED Traffic Signal

Objective and Hardware Connection Construct a single-lane traffic light simulator incorporating three LEDs: Red on PA5 (LED_RED_Pin), Yellow on PA6, and Green on PA7. The timing adheres to municipal signaling specifications: **Red: 5 seconds**, **Yellow: 2 seconds**, and **Green: 3 seconds** (total cycle $T_{cycle} = 10\text{s}$). The complete schematic is shown on the following dedicated page (Figure 2.2).

State	Interval	PA5 (Red)	PA6 (Yellow)	PA7 (Green)	Operational Meaning
1. Stop	5000 ms	RESET (ON)	SET (OFF)	SET (OFF)	Vehicles halt; cross traffic active
2. Caution	2000 ms	SET (OFF)	RESET (ON)	SET (OFF)	Clearance before transit
3. Go	3000 ms	SET (OFF)	SET (OFF)	RESET (ON)	Transit permitted

Table 2.2: Exercise 2 traffic phase states and timing

Firmware Implementation The sequential progression is implemented directly in the main loop of Lab1_e2/Core/Src/main.c:

```

1  /* Infinite loop */
2  /* USER CODE BEGIN WHILE */
3  while (1)
4  {
5      HAL_GPIO_WritePin(GPIOA, LED_RED_Pin, GPIO_PIN_RESET);
6      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
7      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
8      HAL_Delay(5000);
9      HAL_GPIO_WritePin(GPIOA, LED_RED_Pin, GPIO_PIN_SET);
10     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
11     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
12     HAL_Delay(2000);
13     HAL_GPIO_WritePin(GPIOA, LED_RED_Pin, GPIO_PIN_SET);
14     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
15     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
16     HAL_Delay(3000);
17     /* USER CODE END WHILE */
18
19     /* USER CODE BEGIN 3 */
20 }

```

Listing 2.2: Infinite loop implementation for Exercise 2 (Lab1_e2/Core/Src/main.c)

Operational Behavior The program drives exactly one LED low at any time, cyclically iterating through Red (5 s) → Yellow (2 s) → Green (3 s).

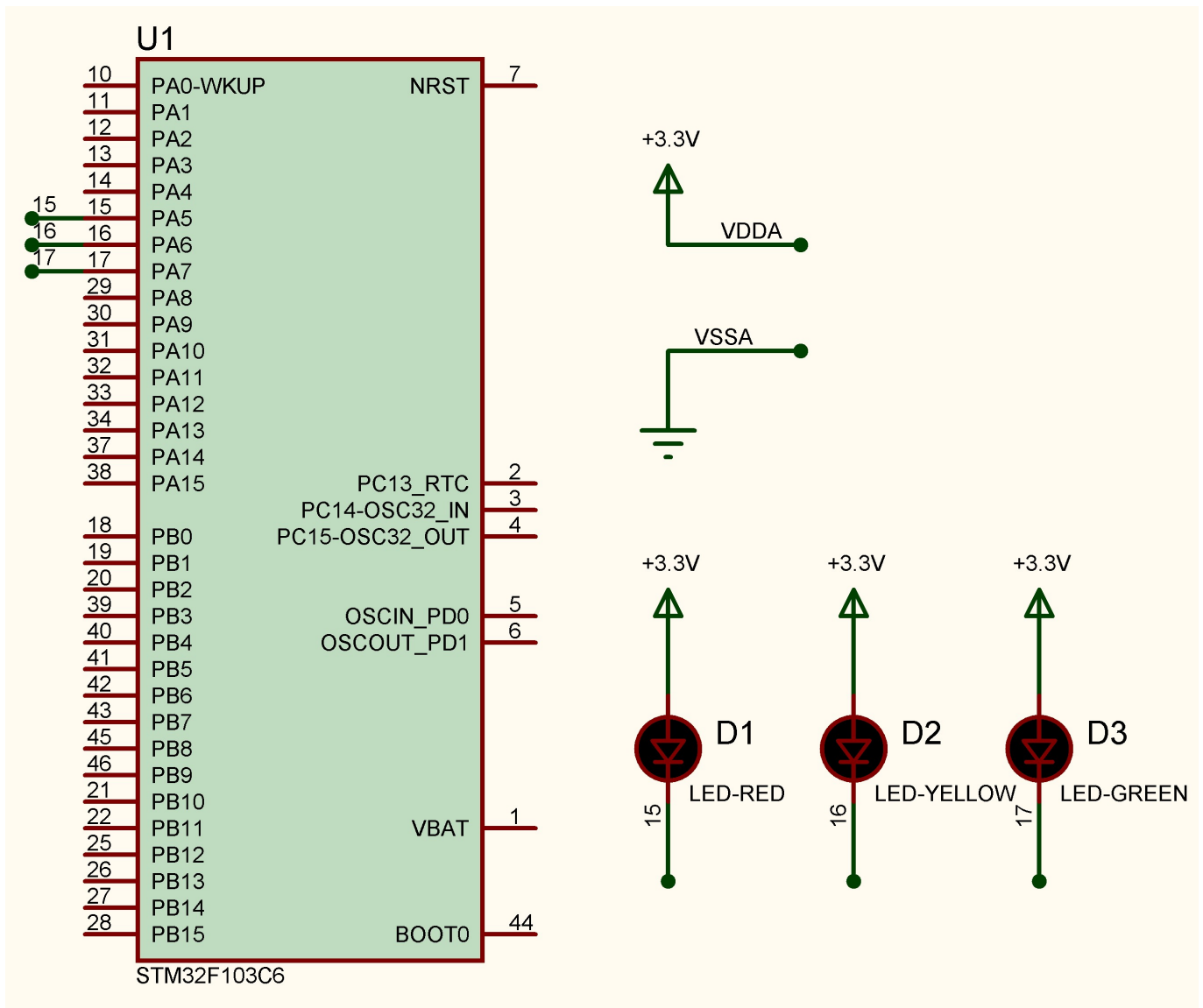


Figure 2.2: Exercise 2 Proteus schematic: Three-LED traffic signal ([Lab1_e2.pdsprj](#))



2.3 Exercise 3: Four-Way 12-LED Traffic Intersection

Objective and Intersecting Geometry Expand the traffic controller to a bidirectional four-way intersection. Twelve LEDs simulate traffic lights across North, South, East, and West approaches. Opposing lanes execute synchronized operations: North and South lanes form the **NS channel**, while East and West lanes form the **WE channel**.

Pin Allocation and Architectural Optimization Rather than consuming 12 separate GPIO pins, each opposing LED pair is wired in parallel to a shared control line, utilizing 6 GPIO outputs on Port A (Table 2.3).

Signal Macro	Microcontroller Pin	Directional Assignment	LED Color
NS_RED_Pin	GPIOA, GPIO_PIN_0	North & South Approaches	Red
NS_YELLOW_Pin	GPIOA, GPIO_PIN_1	North & South Approaches	Yellow
NS_GREEN_Pin	GPIOA, GPIO_PIN_2	North & South Approaches	Green
WE_RED_Pin	GPIOA, GPIO_PIN_3	West & East Approaches	Red
WE_YELLOW_Pin	GPIOA, GPIO_PIN_4	West & East Approaches	Yellow
WE_GREEN_Pin	GPIOA, GPIO_PIN_5	West & East Approaches	Green

Table 2.3: Exercise 3 GPIO pin assignments

Four-Phase State Synthesis To explain the clearance intervals and prevent vehicular collisions, the operational cycle is synthesized into four non-conflicting states (Table 2.4). Note that the lab handout (page 21) introduces the 12-LED layout without mandating a state table; the table is provided here as an analytical explanation of the timing design. The green and yellow durations of one thoroughfare strictly match the red duration of the perpendicular street: $T_{\text{Red}} = T_{\text{Green}} + T_{\text{Yellow}} = 3\text{ s} + 2\text{ s} = 5\text{ s}$.

Phase	Time	North-South (NS)	West-East (WE)	Traffic Flow Status
Phase 1	3 s	GREEN	RED	NS transit active; WE traffic stopped.
Phase 2	2 s	YELLOW	RED	NS clearance period; WE waiting.
Phase 3	3 s	RED	GREEN	WE transit active; NS traffic stopped.
Phase 4	2 s	RED	YELLOW	WE clearance period; NS waiting.

Table 2.4: Four-way traffic light state transition explanation

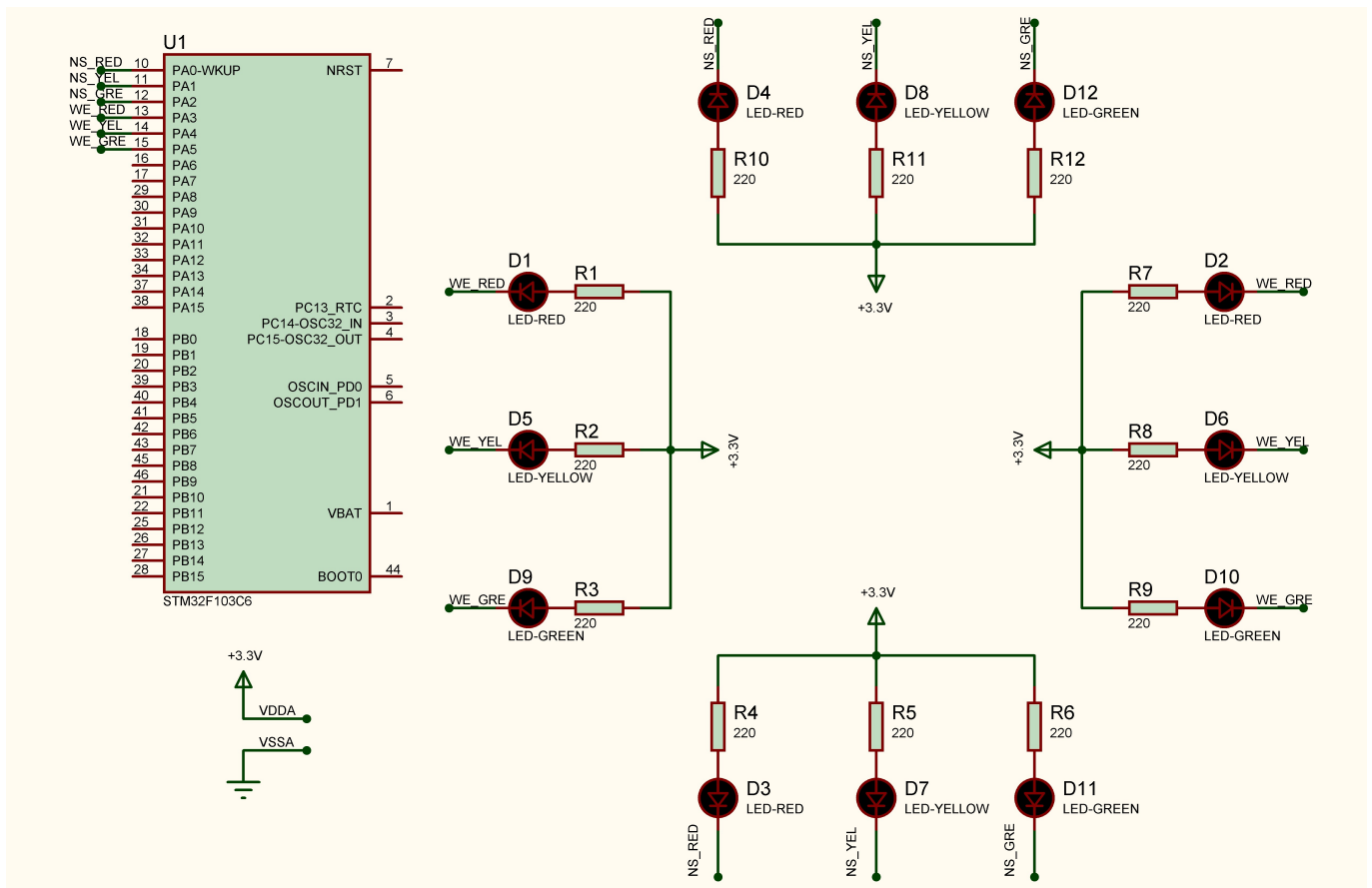


Figure 2.3: Exercise 3 complete Proteus schematic. Download the [Lab1_e3.pdsprj](#) project for a closer view.



2.4 Exercise 4: 7-Segment Common-Anode Display Decoding

Display Architecture and Circuit Setup A single-digit 7-segment display (common-anode 7SEG-COM-ANODE) is integrated into the system. In the Proteus schematic (Figure 2.4, presented in full on the subsequent dedicated page), the common anode terminal is connected to the +3.3V power supply rail. The cathode segment pins (a, b, c, d, e, f, g) are wired to Port B pins PB0 through PB6. The function `display7SEG(int num)` decodes numerical inputs $0 \leq \text{num} \leq 9$.

Active-Low Segment Look-Up Table Because the display is common-anode, pulling a GPIO pin to Logic 0 (GPIO_PIN_RESET) energizes that segment. Table 2.5 lists the segment activations and corresponding bitmask encodings.

Digit	a	b	c	d	e	f	g	Hex Mask	Active Low Pins (RESET)
0	1	1	1	1	1	1	0	0x3F	PB0, PB1, PB2, PB3, PB4, PB5
1	0	1	1	0	0	0	0	0x06	PB1, PB2
2	1	1	0	1	1	0	1	0x5B	PB0, PB1, PB3, PB4, PB6
3	1	1	1	1	0	0	1	0x4F	PB0, PB1, PB2, PB3, PB6
4	0	1	1	0	0	1	1	0x66	PB1, PB2, PB5, PB6
5	1	0	1	1	0	1	1	0x6D	PB0, PB2, PB3, PB5, PB6
6	1	0	1	1	1	1	1	0x7D	PB0, PB2, PB3, PB4, PB5, PB6
7	1	1	1	0	0	0	0	0x07	PB0, PB1, PB2
8	1	1	1	1	1	1	1	0x7F	PB0, PB1, PB2, PB3, PB4, PB5, PB6
9	1	1	1	1	0	1	1	0x6F	PB0, PB1, PB2, PB3, PB5, PB6

Table 2.5: Segment encoding truth table for common-anode 7-segment display

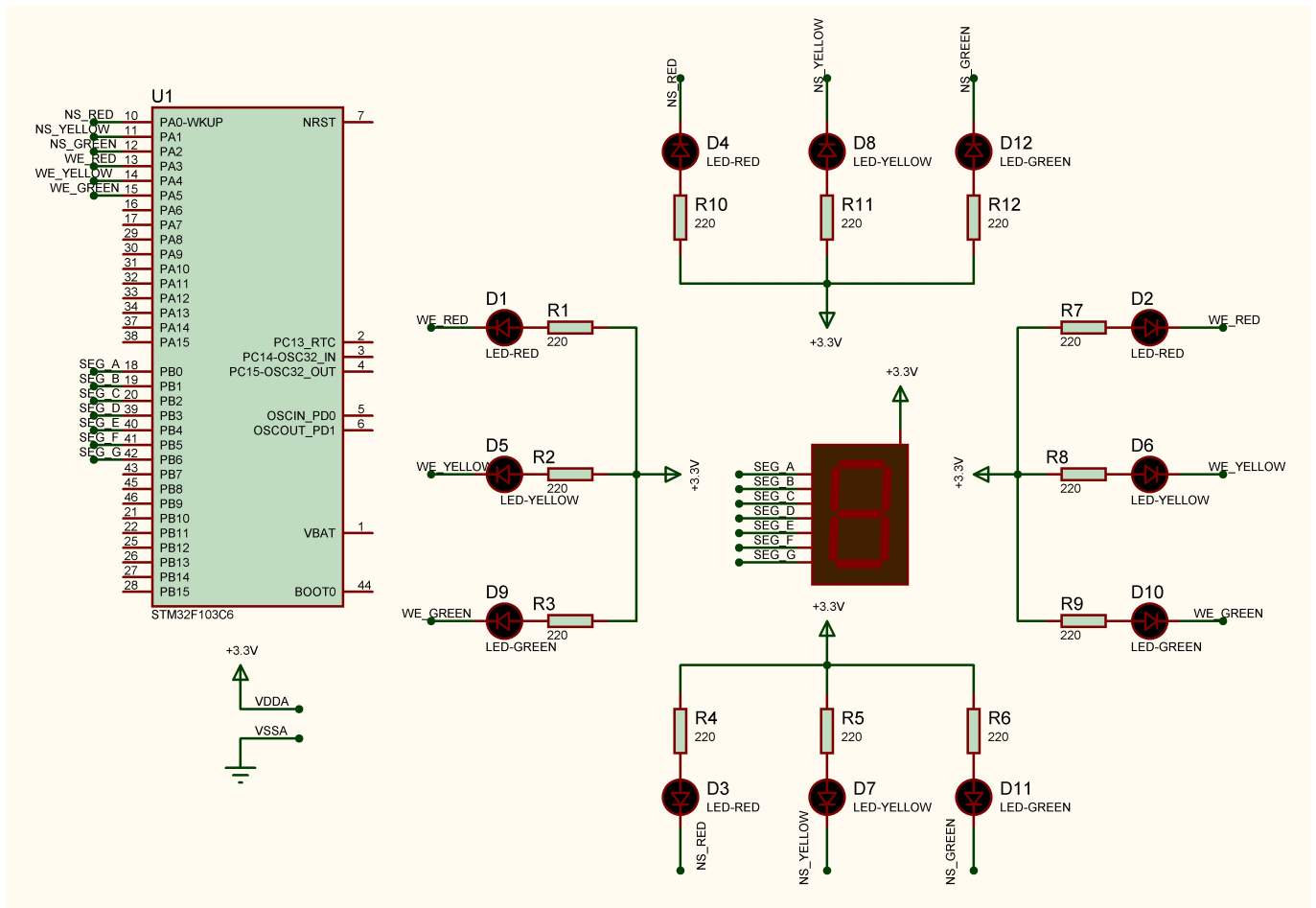


Figure 2.4: Exercise 4 complete Proteus schematic. Download the [Lab1_e4.pdsprj](#) project for a closer view.

Decoding Function Source Code The decoding function applies a bitmask check: if bit n is 1, it emits GPIO_PIN_RESET (ON); otherwise, it emits GPIO_PIN_SET (OFF). The complete verbatim implementation from Lab1_e4/Core/Src/main.c is presented below:

```
1  /* USER CODE BEGIN 0 */
2  void display7SEG(int num)
3  {
4      /*
5       * Bit order:
6       * bit 0 = a
7       * bit 1 = b
8       * bit 2 = c
9       * bit 3 = d
10      * bit 4 = e
11      * bit 5 = f
12      * bit 6 = g
13      *
14      * A 1 in the table means that the segment should be ON.
15      */
16      static const uint8_t digit_pattern[10] =
17      {
18          0x3F,    // 0: a b c d e f
19          0x06,    // 1:  b c
20          0x5B,    // 2: a b  d e  g
21          0x4F,    // 3: a b c d  g
22          0x66,    // 4:  b c  f g
23          0x6D,    // 5: a  c d  f g
24          0x7D,    // 6: a  c d e f g
25          0x07,    // 7: a b c
26          0x7F,    // 8: a b c d e f g
27          0x6F,    // 9: a b c d  f g
28      };
29
30      if (num < 0 || num > 9)
31      {
32          num = 0;
33      }
34
35      uint8_t pattern = digit_pattern[num];
36
37      HAL_GPIO_WritePin(SEG_A_GPIO_Port, SEG_A_Pin,
38          (pattern & (1U << 0)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
39
40      HAL_GPIO_WritePin(SEG_B_GPIO_Port, SEG_B_Pin,
41          (pattern & (1U << 1)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
42
43      HAL_GPIO_WritePin(SEG_C_GPIO_Port, SEG_C_Pin,
44          (pattern & (1U << 2)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
45
46      HAL_GPIO_WritePin(SEG_D_GPIO_Port, SEG_D_Pin,
47          (pattern & (1U << 3)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
48
49      HAL_GPIO_WritePin(SEG_E_GPIO_Port, SEG_E_Pin,
50          (pattern & (1U << 4)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
51
52      HAL_GPIO_WritePin(SEG_F_GPIO_Port, SEG_F_Pin,
53          (pattern & (1U << 5)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
54
55      HAL_GPIO_WritePin(SEG_G_GPIO_Port, SEG_G_Pin,
56          (pattern & (1U << 6)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
57  }
58  /* USER CODE END 0 */
```

Listing 2.3: Complete display7SEG(int num) implementation (Lab1_e4/Core/Src/main.c)



2.5 Exercise 5: Integrated Traffic Light with Countdown Display

Objective and Operational Sequence Integrate the 7-segment display with the four-way traffic controller to present real-time countdown seconds for active traffic phases. In strict accordance with the lab handout (page 22), only the integration source code is required.

Inspection of the actual source implementation in `Lab1_e5/Core/Src/main.c` reveals the exact operational sequence:

- **Phase 1 (3 seconds): North–South RED** (NS_RED_Pin asserted low) and **West–East GREEN** (WE_GREEN_Pin asserted low). The helper function calls `runPhase(3)`, displaying countdown values 3, 2, 1 on the 7-segment display.
- **Phase 2 (2 seconds): North–South RED** (continued for 2 more seconds, completing a 5-second red interval) and **West–East YELLOW** (WE_YELLOW_Pin asserted low). The helper function calls `runPhase(2)`, displaying countdown values 2, 1.
- **Phase 3 (3 seconds): West–East RED** (WE_RED_Pin asserted low) and **North–South GREEN** (NS_GREEN_Pin asserted low). The helper function calls `runPhase(3)`, displaying countdown values 3, 2, 1.
- **Phase 4 (2 seconds): West–East RED** (continued for 2 more seconds, completing a 5-second red interval) and **North–South YELLOW** (NS_YELLOW_Pin asserted low). The helper function calls `runPhase(2)`, displaying countdown values 2, 1.

Verbatim Implementation Source Code The listings below present the complete, verbatim source code from `Lab1_e5/Core/Src/main.c`:

```
1 void runPhase(uint8_t seconds)
2 {
3     for (int time_left = seconds; time_left > 0; time_left--)
4     {
5         display7SEG(time_left);
6         HAL_Delay(1000);
7     }
8 }
9 /* USER CODE END 0 */
```

Listing 2.4: Countdown driver function `runPhase()` (`Lab1_e5/Core/Src/main.c`)

```
1  /* Infinite loop */
2  /* USER CODE BEGIN WHILE */
3  while (1)
4  {
5      /*
6       * Phase 1: NS_RED (3s) & WE_GREEN (3s)
7       * North/South: RED
8       * West/East: GREEN
9       */
10     HAL_GPIO_WritePin(NS_RED_GPIO_Port,
11                       NS_RED_Pin,
12                       GPIO_PIN_RESET);
13
14     HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port,
15                       NS_YELLOW_Pin,
16                       GPIO_PIN_SET);
17
18     HAL_GPIO_WritePin(NS_GREEN_GPIO_Port,
19                       NS_GREEN_Pin,
20                       GPIO_PIN_SET);
21
22     HAL_GPIO_WritePin(WE_RED_GPIO_Port,
23                       WE_RED_Pin,
24                       GPIO_PIN_SET);
25
26     HAL_GPIO_WritePin(WE_YELLOW_GPIO_Port,
27                       WE_YELLOW_Pin,
28                       GPIO_PIN_SET);
29
30     HAL_GPIO_WritePin(WE_GREEN_GPIO_Port,
31                       WE_GREEN_Pin,
32                       GPIO_PIN_RESET);
33
34     runPhase(3);
35
36
37     /*
38      * Phase 2: NS_RED (continued for 2s, total 5s) & WE_YELLOW (2s)
39      * North/South: RED
40      * West/East: YELLOW
41      */
42     HAL_GPIO_WritePin(NS_RED_GPIO_Port,
43                       NS_RED_Pin,
44                       GPIO_PIN_RESET);
45
46     HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port,
47                       NS_YELLOW_Pin,
48                       GPIO_PIN_SET);
49
50     HAL_GPIO_WritePin(NS_GREEN_GPIO_Port,
51                       NS_GREEN_Pin,
52                       GPIO_PIN_SET);
53
54     HAL_GPIO_WritePin(WE_RED_GPIO_Port,
55                       WE_RED_Pin,
56                       GPIO_PIN_SET);
57
58     HAL_GPIO_WritePin(WE_YELLOW_GPIO_Port,
59                       WE_YELLOW_Pin,
60                       GPIO_PIN_RESET);
61
62     HAL_GPIO_WritePin(WE_GREEN_GPIO_Port,
63                       WE_GREEN_Pin,
64                       GPIO_PIN_SET);
65
66     runPhase(2);
67
68
69     /*
70      * Phase 3: WE_RED (3s) & NS_GREEN (3s)
71      * North/South: GREEN
72      * West/East: RED
73      */
74     HAL_GPIO_WritePin(NS_RED_GPIO_Port,
75                       NS_RED_Pin,
76                       GPIO_PIN_SET);
77
```

```
78     HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port ,
79                         NS_YELLOW_Pin ,
80                         GPIO_PIN_SET);
81
82     HAL_GPIO_WritePin(NS_GREEN_GPIO_Port ,
83                         NS_GREEN_Pin ,
84                         GPIO_PIN_RESET);
85
86     HAL_GPIO_WritePin(WE_RED_GPIO_Port ,
87                         WE_RED_Pin ,
88                         GPIO_PIN_RESET);
89
90     HAL_GPIO_WritePin(WE_YELLOW_GPIO_Port ,
91                         WE_YELLOW_Pin ,
92                         GPIO_PIN_SET);
93
94     HAL_GPIO_WritePin(WE_GREEN_GPIO_Port ,
95                         WE_GREEN_Pin ,
96                         GPIO_PIN_SET);
97
98     runPhase(3);
99
100
101     /*
102     * Phase 4: WE_RED (continued for 2s, total 5s) & NS_YELLOW (2s)
103     * North/South: YELLOW
104     * West/East: RED
105     */
106     HAL_GPIO_WritePin(NS_RED_GPIO_Port ,
107                         NS_RED_Pin ,
108                         GPIO_PIN_SET);
109
110     HAL_GPIO_WritePin(NS_YELLOW_GPIO_Port ,
111                         NS_YELLOW_Pin ,
112                         GPIO_PIN_RESET);
113
114     HAL_GPIO_WritePin(NS_GREEN_GPIO_Port ,
115                         NS_GREEN_Pin ,
116                         GPIO_PIN_SET);
117
118     HAL_GPIO_WritePin(WE_RED_GPIO_Port ,
119                         WE_RED_Pin ,
120                         GPIO_PIN_RESET);
121
122     HAL_GPIO_WritePin(WE_YELLOW_GPIO_Port ,
123                         WE_YELLOW_Pin ,
124                         GPIO_PIN_SET);
125
126     HAL_GPIO_WritePin(WE_GREEN_GPIO_Port ,
127                         WE_GREEN_Pin ,
128                         GPIO_PIN_SET);
129
130     runPhase(2);
131 }
132 /* USER CODE END WHILE */
```

Listing 2.5: Integrated traffic countdown while (1) loop (Lab1_e5/Core/Src/main.c)

Operational Behavior The implementation delivers a continuous, repeating 10-second traffic cycle. In Phase 1, West–East traffic proceeds under green while North–South halts; the countdown decrements $3 \rightarrow 2 \rightarrow 1$. In Phase 2, West–East warns vehicles with yellow ($2 \rightarrow 1$) while North–South remains red. In Phase 3, North–South proceeds under green ($3 \rightarrow 2 \rightarrow 1$) while West–East stops. In Phase 4, North–South clears under yellow ($2 \rightarrow 1$) while West–East remains red. All phase durations and display outputs match the C code logic.



3 Analog Clock Simulation with 12 LEDs (Exercises 6–10)

3.1 Exercise 6: 12-LED Clock Dial Interface and Sequential Test

Objective and Hardware Setup Simulate an analog clock dial using 12 discrete LEDs arranged radially. In the Proteus schematic (Figure 3.1, presented in full on the subsequent dedicated page), the LEDs connect to Port A pins PA4 through PA15. A sequential testing program verifies the electrical connections by turning on each individual LED in sequence with a 500 ms dwell time.

Dial LED Pin Allocations and Sequential Sweep Order Table 3.1 details the radial LED connection topology across Port A and the activation sequence executed by the testing firmware.

Dial Numeral	LED Designator	MCU Port Pin	Sequential Verification Step
12	D1	GPIOA, GPIO_PIN_4	Step 1: Active-low pulse (500 ms)
1	D2	GPIOA, GPIO_PIN_5	Step 2: Active-low pulse (500 ms)
2	D3	GPIOA, GPIO_PIN_6	Step 3: Active-low pulse (500 ms)
3	D4	GPIOA, GPIO_PIN_7	Step 4: Active-low pulse (500 ms)
4	D5	GPIOA, GPIO_PIN_8	Step 5: Active-low pulse (500 ms)
5	D6	GPIOA, GPIO_PIN_9	Step 6: Active-low pulse (500 ms)
6	D7	GPIOA, GPIO_PIN_10	Step 7: Active-low pulse (500 ms)
7	D8	GPIOA, GPIO_PIN_11	Step 8: Active-low pulse (500 ms)
8	D9	GPIOA, GPIO_PIN_12	Step 9: Active-low pulse (500 ms)
9	D10	GPIOA, GPIO_PIN_13	Step 10: Active-low pulse (500 ms)
10	D11	GPIOA, GPIO_PIN_14	Step 11: Active-low pulse (500 ms)
11	D12	GPIOA, GPIO_PIN_15	Step 12: Active-low pulse (500 ms)

Table 3.1: Exercise 6 dial LED pin assignments and sweep sequence

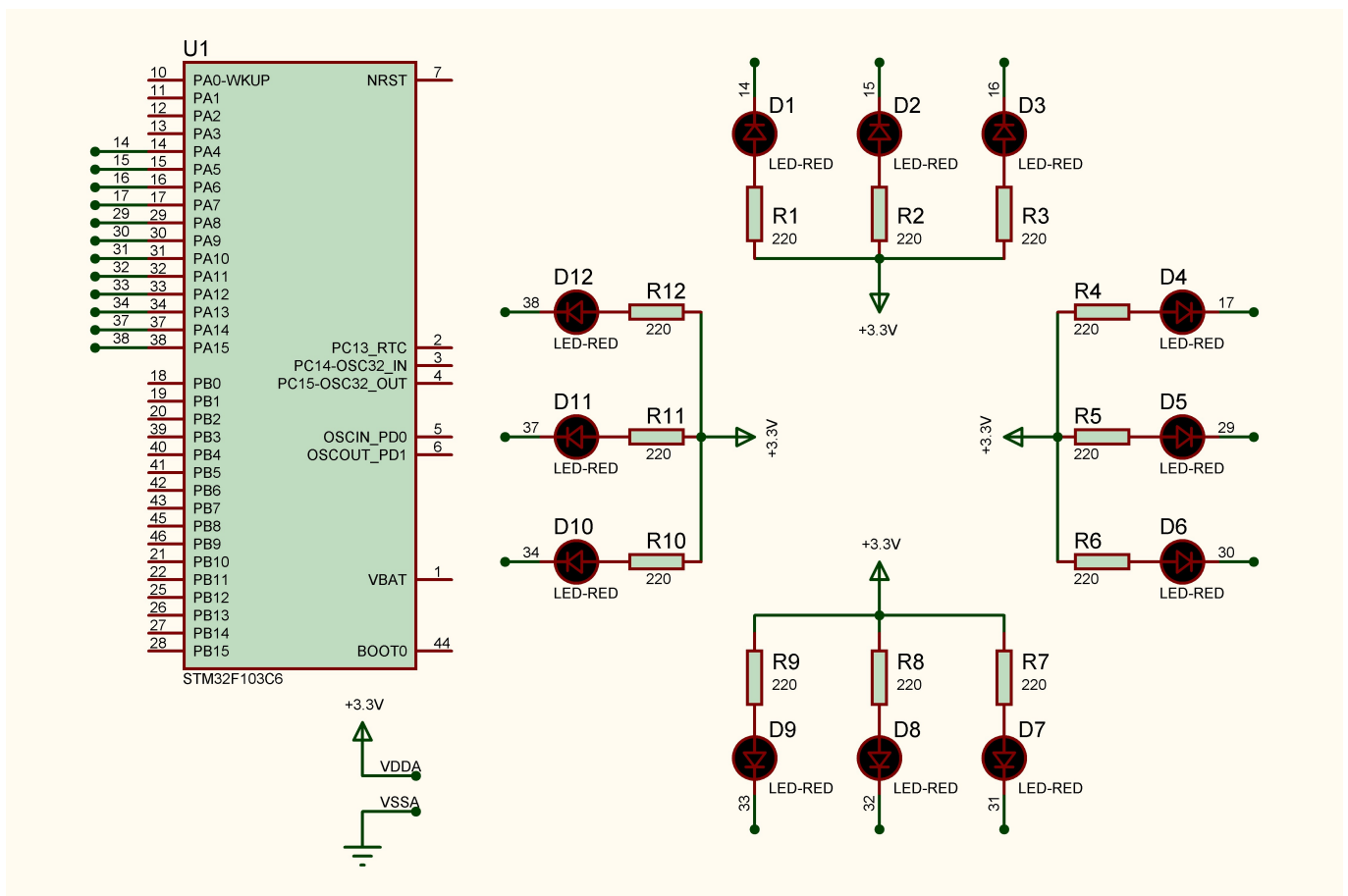


Figure 3.1: Exercise 6 complete Proteus schematic. Download the [Lab1_e6.pdsprj](#) project for a closer view.

Sequential Test Program Implementation The listing below reproduces the verbatim implementation from Lab1_e6/Core/Src/main.c. Rather than using an array loop, the source code executes explicit sequential HAL writes: first setting all 12 pins high (GPIO_PIN_SET) to extinguish the display, then pulsing each pin low (GPIO_PIN_RESET) for 500 ms:

```
1  /* Infinite loop */
2  /* USER CODE BEGIN WHILE */
3  while (1)
4  {
5      // Turn all 12 LEDs OFF
6      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);
7      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
8      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
9      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
10     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
11     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);
12     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET);
13     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET);
14     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET);
15     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET);
16     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET);
17     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_SET);
18
19     // Turn on one LED at a time
20     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET);
21     HAL_Delay(500);
22
23     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);
24     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
25     HAL_Delay(500);
26
27     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
28     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
29     HAL_Delay(500);
30
31     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
32     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
33     HAL_Delay(500);
34
35     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
36     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
37     HAL_Delay(500);
38
39     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
40     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
41     HAL_Delay(500);
42
43     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);
44     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET);
45     HAL_Delay(500);
46
47     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET);
48     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET);
49     HAL_Delay(500);
50
51     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET);
52     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_RESET);
53     HAL_Delay(500);
54
55     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET);
56     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET);
57     HAL_Delay(500);
58
59     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET);
60     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET);
61     HAL_Delay(500);
62
63     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET);
64     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_RESET);
65     HAL_Delay(500);
66
67     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_SET);
68 }
69 /* USER CODE END WHILE */
```

Listing 3.1: Sequential LED test program (Lab1_e6/Core/Src/main.c)

3.2 Exercise 7: Complete clearAllClock() Implementation

Objective and Helper Logic Implement a dedicated helper function to clear all 12 clock LEDs simultaneously. In active-low configuration, setting the pin to Logic 1 opens the circuit by equalizing anode and cathode potentials. The listing below presents the verbatim implementation from Lab1_e7/Core/Src/main.c:

```
1  /* USER CODE BEGIN 0 */
2  void clearAllClock(void)
3  {
4      HAL_GPIO_WritePin(
5          GPIOA,
6          GPIO_PIN_4 |
7          GPIO_PIN_5 |
8          GPIO_PIN_6 |
9          GPIO_PIN_7 |
10         GPIO_PIN_8 |
11         GPIO_PIN_9 |
12         GPIO_PIN_10 |
13         GPIO_PIN_11 |
14         GPIO_PIN_12 |
15         GPIO_PIN_13 |
16         GPIO_PIN_14 |
17         GPIO_PIN_15,
18         GPIO_PIN_SET
19     );
20 }
21 /* USER CODE END 0 */
```

Listing 3.2: Complete clearAllClock() implementation (Lab1_e7/Core/Src/main.c)

By performing a bitwise OR of pin bitmasks 0x0010 through 0x8000 (resulting in 0xFFFF0), the HAL driver updates all 12 pins in a single register write to GPIOA->BSRR.

3.3 Exercise 8: Complete setNumberOnClock(int num) Implementation

Objective and Implementation Implement setNumberOnClock(int num) to selectively turn on the single LED corresponding to input index $0 \leq \text{num} \leq 11$. The function clears all other LEDs prior to asserting the target pin low. The listing below presents the verbatim implementation from Lab1_e8/Core/Src/main.c:

```
1  /* USER CODE BEGIN 0 */
2  void setNumberOnClock(int num)
3  {
4      static const uint16_t clock_pins[12] =
5      {
6          GPIO_PIN_4,
7          GPIO_PIN_5,
8          GPIO_PIN_6,
9          GPIO_PIN_7,
10         GPIO_PIN_8,
11         GPIO_PIN_9,
12         GPIO_PIN_10,
13         GPIO_PIN_11,
14         GPIO_PIN_12,
15         GPIO_PIN_13,
16         GPIO_PIN_14,
17         GPIO_PIN_15
18     };
19
20     // Turn off all LEDs first
21     clearAllClock();
22
23     // Accept only numbers from 0 to 11
```

```
24     if (num >= 0 && num < 12)
25     {
26         // Active-low LED: RESET turns the selected LED on
27         HAL_GPIO_WritePin(GPIOA,
28                             clock_pins[num],
29                             GPIO_PIN_RESET);
30     }
31 }
32 /* USER CODE END 0 */
```

Listing 3.3: Complete setNumberOnClock(int num) implementation
(Lab1_e8/Core/Src/main.c)

3.4 Exercise 9: Complete clearNumberOnClock(int num) Implementation

Objective and Implementation Construct clearNumberOnClock(int num) to selectively turn off the single LED at index $0 \leq \text{num} \leq 11$ without altering other dial outputs. The listing below reproduces the implementation from Lab1_e9/Core/Src/main.c:

```
1  /* USER CODE BEGIN PV */
2  void clearNumberOnClock(int num)
3  {
4      static const uint16_t clock_pins[12] =
5      {
6          GPIO_PIN_4,
7          GPIO_PIN_5,
8          GPIO_PIN_6,
9          GPIO_PIN_7,
10         GPIO_PIN_8,
11         GPIO_PIN_9,
12         GPIO_PIN_10,
13         GPIO_PIN_11,
14         GPIO_PIN_12,
15         GPIO_PIN_13,
16         GPIO_PIN_14,
17         GPIO_PIN_15
18     };
19
20     // Accept only numbers from 0 to 11
21     if (num >= 0 && num < 12)
22     {
23         /*
24          * Active-low LEDs:
25          * GPIO_PIN_SET turns the selected LED OFF.
26          */
27         HAL_GPIO_WritePin(GPIOA,
28                             clock_pins[num],
29                             GPIO_PIN_SET);
30     }
31 }
32 /* USER CODE END PV */
```

Listing 3.4: Complete clearNumberOnClock(int num) implementation
(Lab1_e9/Core/Src/main.c)

3.5 Exercise 10: Integrated Real-Time 12-LED Analog Clock

Clock Face Geometry and Physical Pin Mapping In Exercise 10, the radial arrangement physically corresponds to 12 clock hour positions. Detailed netlist inspection of the Pro-

teus schematic (Lab1_e10.pdsprj) reveals the mapping between clock dial numerals, diode component designators, and STM32 GPIO pins (Table 3.2).

Clock Numerals	Dial Position	Diode Designator	Port Pin	Bitmask Value
12 o'clock	Position 0	D2	PA5	0x0020
1 o'clock	Position 1	D3	PA6	0x0040
2 o'clock	Position 2	D4	PA7	0x0080
3 o'clock	Position 3	D5	PA8	0x0100
4 o'clock	Position 4	D6	PA9	0x0200
5 o'clock	Position 5	D7	PA10	0x0400
6 o'clock	Position 6	D8	PA11	0x0800
7 o'clock	Position 7	D9	PA12	0x1000
8 o'clock	Position 8	D10	PA13	0x2000
9 o'clock	Position 9	D11	PA14	0x4000
10 o'clock	Position 10	D12	PA15	0x8000
11 o'clock	Position 11	D1	PA4	0x0010

Table 3.2: Clock dial numeral to STM32 GPIO pin mapping

Note that **12 o'clock corresponds to D2 (PA5)**, while **11 o'clock corresponds to D1 (PA4)**. This mapping is embedded directly in `clock_pins[12]` in `main.c`.

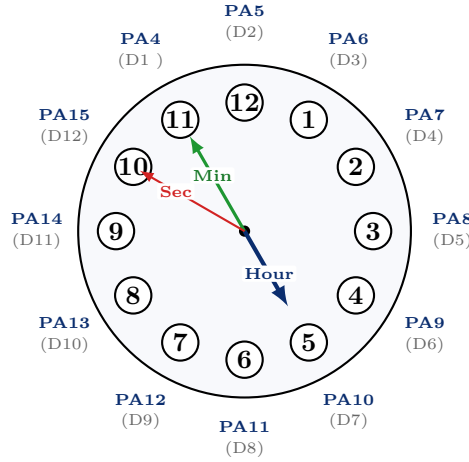


Figure 3.2: 12-LED analog clock dial schematic and hand vector representation

Mathematical Hand Discretization The hour, minute, and second values are quantized into 12 discrete angular sectors:

$$\text{pos}_{\text{hour}} = \text{hour} \pmod{12}, \quad \text{pos}_{\text{minute}} = \lfloor \text{minute}/5 \rfloor, \quad \text{pos}_{\text{second}} = \lfloor \text{second}/5 \rfloor \quad (3.1)$$

Overlapping Hands and Pairwise Distinct Conditions A fundamental physical property arises from mapping three continuous time hands onto 12 shared discrete indicators:

- **Three Distinct Positions:** Exactly **3 distinct LEDs** are illuminated if and only if the three hand positions are pairwise distinct:

$$\text{pos}_{\text{hour}} \neq \text{pos}_{\text{minute}} \quad \text{and} \quad \text{pos}_{\text{minute}} \neq \text{pos}_{\text{second}} \quad \text{and} \quad \text{pos}_{\text{hour}} \neq \text{pos}_{\text{second}} \quad (3.2)$$

- **Two Hands Coincide:** If exactly two hands occupy the same 5-minute sector (e.g., at 12 : 00 : 15, $\text{pos}_{\text{hour}} = 0$ and $\text{pos}_{\text{minute}} = 0$, while $\text{pos}_{\text{second}} = 3$), the bitwise OR union excites only 2 pins, so **exactly 2 distinct LEDs** are illuminated.
- **All Three Hands Coincide:** At times such as 01 : 05 : 05 (or 12 : 00 : 00), all three hands evaluate to the exact same position ($\text{pos}_{\text{hour}} = 1 \pmod{12} = 1$, $\text{pos}_{\text{minute}} = \lfloor 5/5 \rfloor = 1$, $\text{pos}_{\text{second}} = \lfloor 5/5 \rfloor = 1$). The bitwise union yields a single active pin, illuminating **only 1 distinct LED**.

Therefore, three time values correspond to **fewer than three distinct illuminated LEDs** whenever two or more hands overlap.

Atomic Port Manipulation via BSRR In `displayClock()`, active LED bitmasks combine with bitwise OR, and inactive LEDs are isolated:

$$\text{on_pins} = \text{clock_pins}[\text{pos}_{\text{hour}}] \mid \text{clock_pins}[\text{pos}_{\text{minute}}] \mid \text{clock_pins}[\text{pos}_{\text{second}}] \quad (3.3)$$

$$\text{off_pins} = \text{CLOCK_LED_MASK} \& (\sim \text{on_pins}) \quad (3.4)$$

The entire 12-pin port configuration is applied in a single atomic instruction to the Bit Set/Reset Register^[1]:

$$\text{GPIOA} \rightarrow \text{BSRR} = (\text{uint32_t})\text{off_pins} \mid ((\text{uint32_t})\text{on_pins} \ll 16) \quad (3.5)$$

Because the BSRR high-half resets pins low (turning LEDs ON) and low-half sets pins high (turning LEDs OFF) simultaneously, the clock updates without intermediate glitch states.

Verbatim Clock Implementation Source Code The listings below present the verbatim display driver and real-time execution loop from `Lab1_e10/Core/Src/main.c`:

```
1  #define CLOCK_LED_MASK ((uint16_t)0xFFFF0U)
2
3  static const uint16_t clock_pins[12] =
4  {
5      GPIO_PIN_5,
6      GPIO_PIN_6,
7      GPIO_PIN_7,
8      GPIO_PIN_8,
9      GPIO_PIN_9,
10     GPIO_PIN_10,
11     GPIO_PIN_11,
12     GPIO_PIN_12,
13     GPIO_PIN_13,
14     GPIO_PIN_14,
15     GPIO_PIN_15,
16     GPIO_PIN_4
17 };
18
19 static void displayClock(uint8_t hour, uint8_t minute, uint8_t second)
20 {
21     const uint16_t on_pins = clock_pins[hour % 12U]
22                             | clock_pins[minute / 5U]
23                             | clock_pins[second / 5U];
24     const uint16_t off_pins = CLOCK_LED_MASK & (uint16_t)~on_pins;
```

```
25
26  /* Update all hands together; leave PA0-PA3 unchanged.
27  * Low half sets OFF pins high; high half resets ON pins low.
28  * Overlapping hands naturally share the same LED.
29  */
30  GPIOA->BSRR = (uint32_t)off_pins | ((uint32_t)on_pins << 16U);
31 }
```

Listing 3.5: Clock display and atomic BSRR driver (Lab1_e10/Core/Src/main.c)

```
1  /* Initial time: 05:59:50 */
2  uint8_t hour = 5;
3  uint8_t minute = 59;
4  uint8_t second = 50;
5  uint32_t previous_tick = HAL_GetTick();
6
7  displayClock(hour, minute, second);
8
9  while (1)
10 {
11     /* Unsigned subtraction also handles the HAL tick wrapping.
12     * Fixed deadlines avoid adding loop execution time each second.
13     */
14     while ((uint32_t)(HAL_GetTick() - previous_tick) >= 1000U)
15     {
16         previous_tick += 1000U;
17         if (++second >= 60U)
18         {
19             second = 0;
20             if (++minute >= 60U)
21             {
22                 minute = 0;
23                 if (++hour > 12U)
24                 {
25                     hour = 1;
26                 }
27             }
28         }
29         displayClock(hour, minute, second);
30     }
31     HAL_Delay(1);
32 }
```

Listing 3.6: Real-time tick loop with rollover management (Lab1_e10/Core/Src/main.c)

Time-Base Generation and Rollover Execution Starting at 05 : 59 : 50, the system executes for 10 seconds, reaching a cascading rollover: at second 60, `second` resets to 0, `minute` rolls from 59 to 0, and `hour` advances from 5 to 6 (06 : 00 : 00). At 06 : 00 : 00, the hour hand maps to position 6 (PA11), while minute and second hands coincide at position 0 (PA5), illuminating exactly 2 distinct LEDs. This behavior is established from source analysis.

4 Repository Artifact Links

Each design module is committed in the project repository:

- Ex 1 Proteus Project: [Proteus_Project/Lab1_e1/Lab1_e1.pdsprj](#)
- Ex 2 Proteus Project: [Proteus_Project/Lab1_e2/Lab1_e2.pdsprj](#)
- Ex 3 Proteus Project: [Proteus_Project/Lab1_e3/Lab1_e3.pdsprj](#)
- Ex 4 Proteus Project: [Proteus_Project/Lab1_e4/Lab1_e4.pdsprj](#)

- Ex 5 Proteus Project: [Proteus_Project/Lab1_e5/Lab1_e5.pdsprj](#)
- Ex 6 Proteus Project: [Proteus_Project/Lab1_e6/Lab1_e6.pdsprj](#)
- Ex 10 Proteus Project: [Proteus_Project/Lab1_e10/Lab1_e10.pdsprj](#)

References

- [1] STMicroelectronics, *RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced Arm-based 32-bit MCUs*, Rev. 21, 2021.
- [2] STMicroelectronics, *STM32F103x4, STM32F103x6 Low-Density Performance Line ARM-Based 32-Bit MCU Datasheet*, DocID 15060 Rev. 10, 2020.
- [3] STMicroelectronics, *UM1850 User Manual: Description of STM32F1 HAL and low-layer drivers*, Rev. 5, 2021.
- [4] Le Trong Nhan, *Lab 01: LED Animations — Microcontroller Laboratory Manual*, Department of Computer Engineering, Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), pp. 20–23.
- [5] Labcenter Electronics Ltd., *Proteus VSM for ARM Cortex-M3 Simulation Guide*, Version 8.13, 2021.
- [6] N. M. Phuc, *Microcontroller Course Laboratory Code Repository*, GitHub repository, commit 77b62e4, 2026. <https://github.com/tapstoptap/microcontroller-261.git>.